# Assignment #A: Graph starts

Updated 1830 GMT+8 Apr 22, 2025

2025 spring, Complied by 李皓翔 物院

> **说明：**
>
> 1. **解题与记录：**
>
>    对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora https://typoraio.cn 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。
>
> 2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
>
> 3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。
>
> 请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

## 1. 题目

### M19943:图的拉普拉斯矩阵

OOP, implementation, http://cs101.openjudge.cn/practice/19943/

要求创建Graph, Vertex两个类，建图实现。

思路：

代码：

```
class Vertex:
    def __init__(self,key):
        self.key=key
        self.neighbors=[]
    def set_neighbor(self,p):
        self.neighbors.append(p)

class Graph:
    def __init__(self):
        self.vertices={}

    def set_vertex(self,key):
        self.vertices[key]=Vertex(key)

    def add_edge(self,a,b):
```

```
        if a not in self.vertices:
            self.set_vertex(a)
        if b not in self.vertices:
            self.set_vertex(b)
        self.vertices[a].set_neighbor(self.vertices[b])
        self.vertices[b].set_neighbor(self.vertices[a])

    def D_out(self):
        tem=[[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            tem[i][i]=len(self.vertices[i].neighbors)
        return tem

    def A_out(self):
        tem=[[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(i+1):
                if self.vertices[i] in self.vertices[j].neighbors:
                    tem[i][j]=1
                    tem[j][i]=1
        return tem


n,m=map(int,input().split())
g=Graph()
for i in range(n):
    g.set_vertex(i)

for _ in range(m):
    a,b=map(int,input().split())
    g.add_edge(a,b)

A=g.A_out()
D=g.D_out()
L=[[0 for _ in range(n)] for _ in range(n)]

for i in range(n):
    for j in range(n):
        L[i][j]=D[i][j]-A[i][j]

for i in L:
    print(' '.join(list(map(str,i))))
```

代码运行截图 （至少包含有"Accepted"）

**#49033531提交状态**

状态: Accepted

源代码

```
class Vertex:
    def __init__(self,key):
        self.key=key
        self.neighbors=[]
    def set_neighbor(self,p):
        self.neighbors.append(p)

class Graph:
    def __init__(self):
        self.vertices={}

    def set_vertex(self,key):
        self.vertices[key]=Vertex(key)

    def add_edge(self,a,b):
        if a not in self.vertices:
            self.set_vertex(a)
        if b not in self.vertices:
            self.set_vertex(b)
        self.vertices[a].set_neighbor(self.vertices[b])
        self.vertices[b].set_neighbor(self.vertices[a])

    def D_out(self):
        tem=[[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            tem[i][i]=len(self.vertices[i].neighbors)
        return tem

    def A_out(self):
        tem=[[0 for _ in range(n)] for _ in range(n)]
```

## LC78.子集

backtracking, https://leetcode.cn/problems/subsets/

思路:

代码:

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i,p,ans):
            if i==n:
                ans.append(list(p))
                return ans
            ans=dfs(i+1,p,ans)
            p.append(nums[i])
            ans=dfs(i+1,p,ans)
            p.pop()
```

```
            return ans

        n=len(nums)
        ans=dfs(0,[],[])
        return ans
```

代码运行截图 （至少包含有"Accepted"）


## LC17.电话号码的字母组合

hash table, backtracking, https://leetcode.cn/problems/letter-combinations-of-a-phone-number/

思路:

代码:

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if digits=='':
            return []
        ht={'2':['a','b','c'],
                '3':['d','e','f'],
```

```
                '4':['g','h','i'],
                '5':['j','k','l'],
                '6':['m','n','o'],
                '7':['p','q','r','s'],
                '8':['t','u','v'],
                '9':['w','x','y','z']}
        n=len(digits)

        def dfs(i,ans,p):
            if i==n:
                ans.append(str(''.join(p)))
                return ans
            for j in ht[digits[i]]:
                p.append(j)
                ans=dfs(i+1,ans,p)
                p.pop()
            return ans

        ans=dfs(0,[],[])
        return ans
```

代码运行截图（至少包含有"Accepted"）

# M04089:电话号码

trie, http://cs101.openjudge.cn/practice/04089/

思路:

代码:

```
class TreeNode:
    def __init__(self,value):
        self.value=value
        self.child={}
        self.end=False

for _ in range(int(input())):
    trie=TreeNode('start')
    flag=True
    for _ in range(int(input())):
        tem=input()
        if not flag:
            continue
        n=len(tem)
        pre=trie
        t=False
        for i in range(n):
            if pre.end:
                break
            if tem[i] in pre.child:
                pre=pre.child[tem[i]]
            else:
                pre.child[tem[i]]=TreeNode(tem[i])
                t=True
                pre=pre.child[tem[i]]
        pre.end=True
        if not t:
            flag=False
    if flag:
        print('YES')
    else:
        print('NO')
```

代码运行截图 （至少包含有"Accepted"）

## 状态：Accepted

源代码

```
class TreeNode:
    def __init__(self,value):
        self.value=value
        self.child={}
        self.end=False

for _ in range(int(input())):
    trie=TreeNode('start')
    flag=True
    for _ in range(int(input())):
        tem=input()
        if not flag:
            continue
        n=len(tem)
        pre=trie
        t=False
        for i in range(n):
            if pre.end:
                break
            if tem[i] in pre.child:
                pre=pre.child[tem[i]]
            else:
                pre.child[tem[i]]=TreeNode(tem[i])
                t=True
                pre=pre.child[tem[i]]
        pre.end=True
        if not t:
            flag=False
    if flag:
        print('YES')
    else:
        print('NO')
```

## T28046:词梯

bfs, http://cs101.openjudge.cn/practice/28046/

思路：

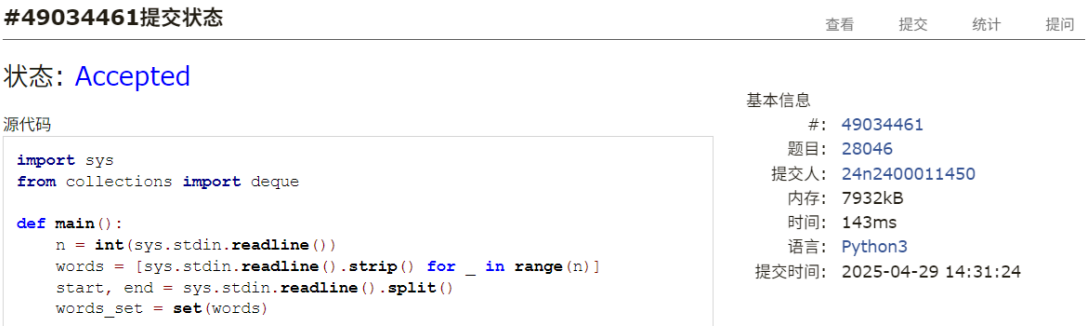
代码：

```
import sys
from collections import deque

def main():
    n = int(sys.stdin.readline())
    words = [sys.stdin.readline().strip() for _ in range(n)]
    start, end = sys.stdin.readline().split()
    words_set = set(words)
```

```
    if start not in words_set or end not in words_set:
        print("NO")
        return

    if start == end:
        print(start)
        return

    if len(words) > 0:
        first_word = words[0]
        alphabet = 'abcdefghijklmnopqrstuvwxyz' if first_word.islower() else
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
    else:
        alphabet = 'abcdefghijklmnopqrstuvwxyz'  # Default case, though n >= 2

    adjacency = {}
    for word in words:
        neighbors = []
        for i in range(4):
            original_char = word[i]
            for c in alphabet:
                if c != original_char:
                    candidate = word[:i] + c + word[i+1:]
                    if candidate in words_set:
                        neighbors.append(candidate)
        adjacency[word] = neighbors

    parent = {}
    visited = set()
    queue = deque([start])
    visited.add(start)
    found = False

    while queue:
        current = queue.popleft()
        if current == end:
            path = []
            node = current
            while node != start:
                path.append(node)
                node = parent[node]
            path.append(start)
            path.reverse()
            print(' '.join(path))
            found = True
            break
        for neighbor in adjacency[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    if not found:
        print("NO")
```

```
if __name__ == "__main__":
    main()
```

代码运行截图 （至少包含有"Accepted"）



#49034461提交状态

查看 提交 统计 提问

状态: Accepted

源代码

```
import sys
from collections import deque

def main():
    n = int(sys.stdin.readline())
    words = [sys.stdin.readline().strip() for _ in range(n)]
    start, end = sys.stdin.readline().split()
    words_set = set(words)
```

基本信息

#: 49034461
题目: 28046
提交人: 24n2400011450
内存: 7932kB
时间: 143ms
语言: Python3
提交时间: 2025-04-29 14:31:24

## T51.N皇后

backtracking, https://leetcode.cn/problems/n-queens/

思路:

代码:

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        res = []
        path = [0] * n
        col = [False] * n
        m = 2 * n
        diag1 = [False] * m
        diag2 = [False] * m

        def dfs(r):
            if r == n:
                res.append(['.' * i + 'Q' + '.' * (n-i-1) for i in path])
                return
            for c in range(n):
                if not col[c] and not diag1[r-c] and not diag2[r+c]:
                    path[r] = c
                    col[c] = diag1[r-c] = diag2[r+c] = True
                    dfs(r + 1)
                    col[c] = diag1[r-c] = diag2[r+c] = False

        dfs(0)
        return res
```

代码运行截图 （至少包含有"Accepted"）


## 2. 学习总结和收获

T28046:词梯 ds优化方法好妙，自己暴力比较O(n^2)的建图优化了好几次都超时:(